

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине «Программное обеспечение мехатронных
и робототехнических систем»

Тема лабораторной работы «Программное обеспечение системы управления мехатронного модуля управления защитной дверью»

Студент группы: КРБО–01–23 Абакумов А. Я.
Балышев Е.Ю.
Егоров М.Ю.
Сгибнев Д.Р.

_____

Преподаватель: ст. преподаватель Морозов А. А.

_____

Работа представлена к защите: «_____» ____2026 г.

Москва 2026

ЦЕЛЬ РАБОТЫ

Получение навыков построения программного обеспечения промышленных систем управления на основе функциональных блоков и конечных автоматов.

ЗАДАНИЕ

Разработать в B&R Automation Studio программу управления приводом защитной двери с асинхронным двигателем, четырьмя датчиками положения и тумблером направления. В рабочем режиме дверь должна двигаться быстро между двумя внутренними датчиками, снижать скорость у краёв и останавливаться по крайним датчикам.

По заданию при запуске предусмотрены поиск исходного положения на малой скорости и мигание индикаторов с частотой 1 Гц. В рабочем режиме индикаторы должны показывать положение двери. Основную логику требуется оформить в виде повторно используемых функциональных блоков.

ОСНОВНАЯ ЧАСТЬ

4.1 Описание действий и использованного аппарата

Проект Lab4.apj создан в B&R Automation Studio 4.5.2.102. В конфигурации Config1 панель-контроллер 4PP065.0571-P74 связана по POWERLINK с модулями X20SL8000 и X20BC0083. Преобразователь 8I64xxxxxxx.00x-1 подключён к X20BT9100 по шине X2X. Тумблер и датчики подключены к X20DI9371, светодиоды — к X20DO9322.

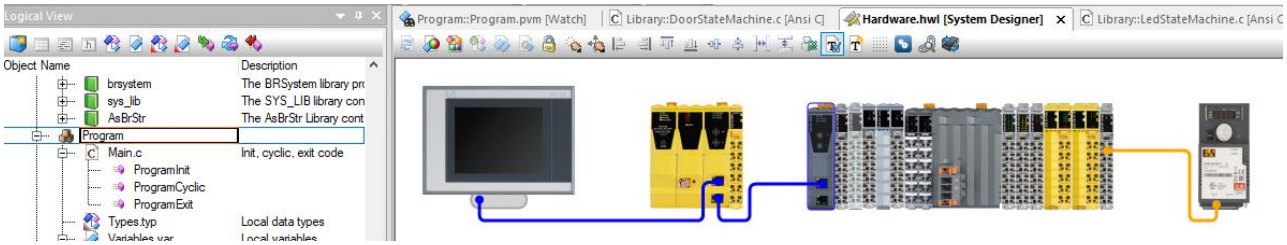


Рисунок 1. Аппаратная конфигурация в окне System Designer

В редакторе конфигурации заданы привязки каналов, логика реализована на ANSI C. Для наблюдения за сигналами использовались окна Watch и Trace. Соответствие каналов приведено в подразделе 4.5.

4.2 Структурная схема конечного автомата

Условные обозначения S_0–S_3 из задания соответствуют входам sw1–sw4. На рисунке 2 показаны рабочие циклы от одного крайнего датчика к другому и переходы при смене направления. Обозначение Δsw означает изменение логического уровня за цикл.

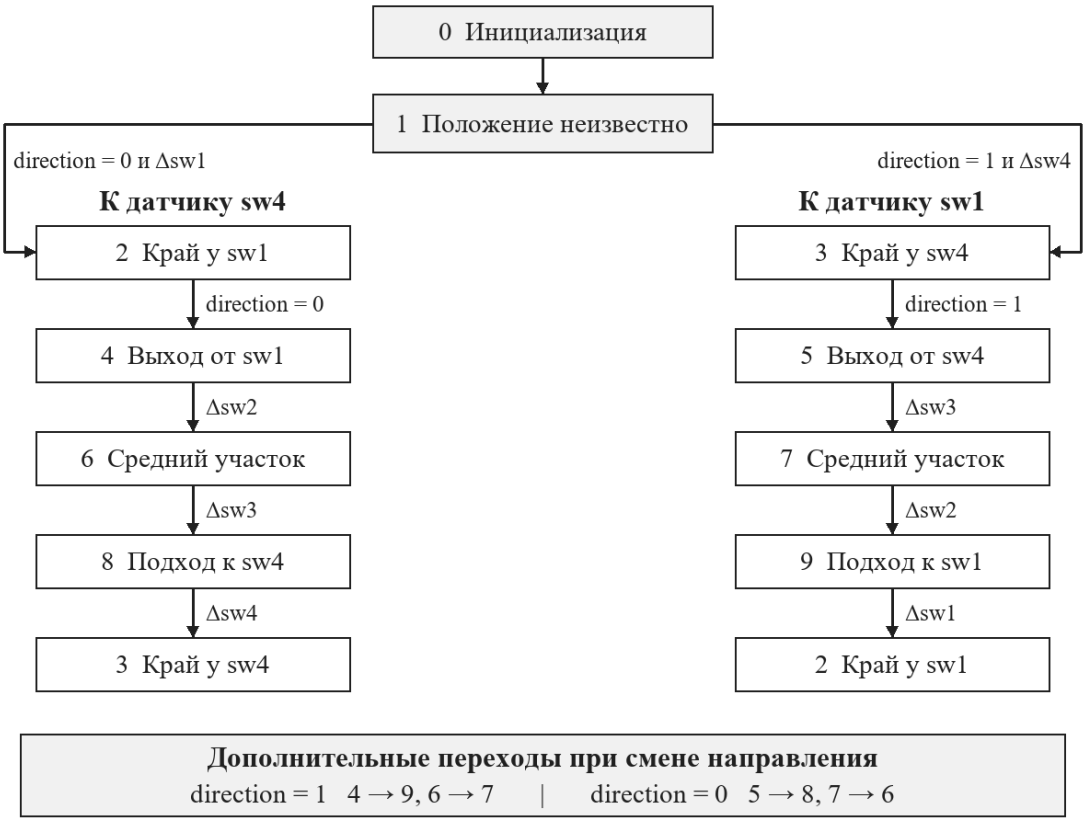


Рисунок 2. Граф переходов блока DoorStateMachine

После обработки входов блок запоминает их уровни в swp1–swp4. Условие swN ≠ swpN выполняется при изменении сигнала в любую сторону. Поэтому алгоритм реагирует и на фронт, и на спад, если соответствующий датчик проверяется в текущем состоянии.

В состояниях 4–7 датчик проверяется раньше тумблера. Если оба события приходятся на один вызов, выполняется переход по датчику. В состояниях 8 и 9 автомат ждёт крайнего датчика.

4.3 Описание состояний

При движении к sw4 уставка положительна, при движении к sw1 — отрицательна. Низкая уставка по модулю равна 250 об/мин, уставка на среднем участке — 750 об/мин. Единица измерения взята из описания канала LFRD_Output в конфигурации.

Таблица 1 – Состояния автомата и соответствующие выходы

Код	Смысл состояния	Уставка, об/мин	led1–led4
0	Первый вызов, переход в состояние 1	0 при старте	[1]
1	Ожидание признака крайнего положения	0 при старте	Мигание
2	Останов у sw1	0	0000
3	Останов у sw4	0	1111
4	Выход от sw1 к sw2	+250	1000
5	Выход от sw4 к sw3	−250	1110
6	Средний участок в сторону sw4	+750	1100
7	Средний участок в сторону sw1	−750	1100
8	Подход к sw4	+250	1110
9	Подход к sw1	−250	1000

[1] В ветви 0 LedStateMachine четыре раза записывает ноль в led1. При штатном запуске Program передаёт в LedState уже обновлённое состояние 1 после первого вызова DoorStateMachine.

В рабочем цикле число горящих светодиодов растёт от sw1 к sw4 и уменьшается при обратном ходе. Комбинации 1000, 1100 и 1110 различают три участка пути. Одинаковые комбинации при ходе туда и обратно обозначают тот же участок.

В состоянии 1 программа ожидает изменение sw1 при direction = 0 либо sw4 при direction = 1. До этого события ветвь поиска сохраняет начальную нулевую уставку. Самостоятельное движение на малой скорости при пуске, предусмотренное заданием, в этой ветви отсутствует.

4.4 Описание функциональных блоков

Пользовательская библиотека Library содержит три блока. Экземпляры DoorState, EVIRD и LedState объявлены в программе Program. Каждый вызов получает указатель на свой экземпляр и изменяет его выходы и внутреннее состояние.

Таблица 2 – Интерфейсы функциональных блоков

Блок	Входы и внутренние данные	Выходы
DoorStateMachine	direction и sw1–sw4 (BOOL). state (INT) хранит код состояния. swp1–swp4 (BOOL) хранят предыдущие уровни.	speed (INT) — уставка скорости, state_out (INT) — копия кода состояния.
DriveStateMachine	ENABLE (BOOL) — разрешение. STATE (UINT) — слово состояния преобразователя.	COM (UINT) — слово команды преобразователя.
LedStateMachine	state (INT) — код от DoorState. timer (UINT) — счётчик вызовов.	led1–led4 (BOOL) — сигналы индикаторов.

Блок DriveStateMachine выделяет значимые биты слова состояния маской 0x6F. При ENABLE = 0 он выдаёт COM = 6. При ENABLE = 1 выбор команды соответствует таблице 3. Для остальных значений маски сохраняется предыдущая команда.

Таблица 3 – Команды преобразователю при ENABLE = 1

STATE & 0x6F	Константа состояния	Команда COM
64	STATE_DISABLED	CMD_SHUTDOWN = 6
33	STATE_READY	CMD_ENABLED = 15
35	STATE_SWITCHED_ON	CMD_SWITCHED_ON = 7

В состоянии 1 блок LedStateMachine синхронно переключает четыре индикатора. При timer = 0…4 они выключены, при 5…9 включены. В отдельном вызове счётчик сбрасывается, а выходы сохраняются. Период установившегося мигания равен 11 периодам вызова блока.

4.5 Структура программного обеспечения

При инициализации ProgramInit устанавливает EVIRD.ENABLE = 1. В циклической задаче Cyclic #4 вызываются DriveStateMachine и DoorStateMachine. Затем код DoorState.state_out передаётся в LedState.state, после чего LedStateMachine обновляет выходы индикаторов.

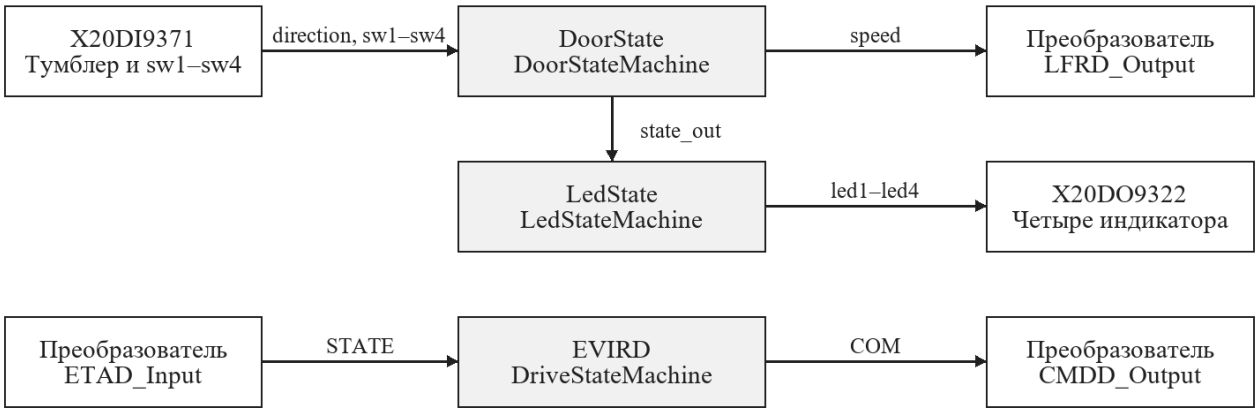


Рисунок 3. Связи функциональных блоков с модулями ввода и вывода

Таблица 4 – Мэпинг сигналов программы Program

Модуль и канал	**Переменная**	**Назначение**
X20DI9371 DigitalInput01	DoorState.direction	Команда направления
X20DI9371 DigitalInput02–05	DoorState.sw1–sw4	Датчики положения по порядку
X20DO9322 DigitalOutput02–05	LedState.led1–led4	Выходы четырёх индикаторов
8I64xxxxxxx.00x-1 ETAD_Input	EVIRD.STATE	Входное слово состояния
8I64xxxxxxx.00x-1 CMDD_Output	EVIRD.COM	Выходное слово команды
8I64xxxxxxx.00x-1 LFRD_Output	DoorState.speed	Выходная уставка скорости

Слова ETAD и CMDD настроены в упакованном виде. Программа обменивается с преобразователем через карту IoMap.iom. Блок двери выдаёт уставку напрямую в LFRD_Output, поэтому блок EVIRD занимается только обработкой слова состояния и выдачей команд.

4.6 Результаты экспериментальных исследований

Trace записывает семь переменных экземпляра DoorState. Сверху вниз показаны direction, state, speed и четыре входа sw1–sw4. Для разбора цикла отслеживались изменения состояния, знак уставки и моменты переключения датчиков.



Рисунок 4. Осциллограмма прохода к sw4 и обратного хода

На рисунке 4 примерно на 18-й секунде начинается движение к sw4. На участке около 21–24 с положительная уставка достигает максимального значения, затем снижается. Около 26 с при изменении sw4 уставка становится нулевой.

Около 29 с команда меняет направление. Уставка принимает отрицательное значение, а при изменении sw3 её модуль растёт. После срабатывания sw2 модуль снижается. В конце участка, около 42 с, изменение sw1 приводит к останову по программе. Времена на этой записи оценены по шкале изображения.

Длительность ступеней определяется порядком и временем поступления сигналов датчиков. Уставка меняется дискретно при переходах автомата, поэтому по графику speed можно проверить алгоритм переключения участков.

Повторные проходы двери

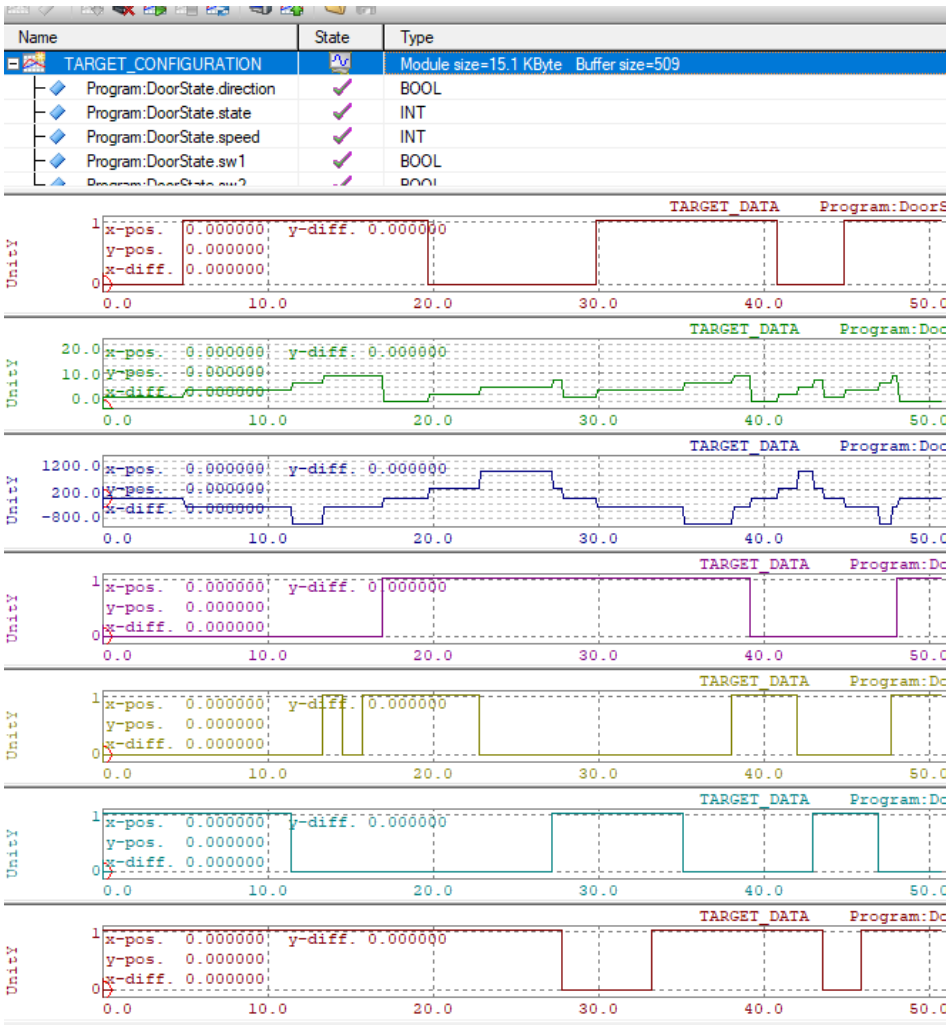


Рисунок 5. Осциллограмма повторных проходов с 509 отсчётами

Для рисунка 5 времена определены по записи Program.td из архива. Шаг записи составляет 0,1 с. В таблице 5 приведены моменты выхода из крайнего состояния и возврата уставки к нулю.

Таблица 5 – Участки движения по записи Program.td

Интервал, с	Движение	Последовательность состояний
5,0–17,0	От sw4 к sw1	3 → 5 → 7 → 9 → 2
19,8–27,9	От sw1 к sw4	2 → 4 → 6 → 8 → 3
30,0–39,3	От sw4 к sw1	3 → 5 → 7 → 9 → 2
40,9–43,7	От sw1 к sw4	2 → 4 → 6 → 8 → 3
45,0–48,2	От sw4 к sw1	3 → 5 → 7 → 9 → 2

На втором проходе уставка менялась от +250 об/мин в 19,8 с до +750 об/мин в 22,9 с после изменения sw2. При изменении sw3 она снизилась до +250 об/мин в 27,3 с. После изменения sw4 уставка вернулась к нулю.

Сопоставление записи с программой

Для сверки исходная функция DoorStateMachine из архива была выполнена отдельно от контроллера. Входы direction и sw1–sw4 подавались в порядке записи Program.td. Полученные state и speed совпали со следующим отсчётом записи во всех 508 сравнениях. Начальное состояние взято из первого отсчёта, где state = 3.

На осциллограммах зарегистрирована командная скорость. Измеренная скорость двигателя, слова ETAD и CMDD и выходы led1–led4 в этих записях отсутствуют. Частота мигания и фактическая отработка уставки приводом требуют отдельных измерений.

ВЫВОДЫ

Автомат формирует уставку ±750 об/мин между внутренними датчиками и снижает её по модулю до 250 об/мин на краевых участках. Изменение крайнего датчика обнуляет уставку. Обе осциллограммы подтверждают этот порядок для двух направлений движения.

Рабочий цикл подтверждён по записанным сигналам. Для выполнения задания при пуске требуется дополнить программу движением на малой скорости в режиме поиска исходного положения.

ИСТОЧНИКИ

1. Методическое пособие по дисциплине «Программное обеспечение мехатронных и робототехнических систем». 2021. Лабораторная работа № 2, с. 12–22.

2. Архив Lab2.zip (проект Lab4.apj, IoMap.iom и запись Program.td). Снимки Automation Studio 11.PNG, 12.PNG и хард.PNG.

ПРИЛОЖЕНИЕ А

Основная программа Program

Program/Main.c

```
#include <bur/plctypes.h>
#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
    #include <stdlib.h>
#endif
void _INIT ProgramInit(void)
{
    EVIRD.ENABLE = 1;
    LedState.state = DoorState.state_out;
}
void _CYCLIC ProgramCyclic(void)
{
    DriveStateMachine(&EVIRD);
    DoorStateMachine(&DoorState);
    LedState.state = DoorState.state_out;
    LedStateMachine(&LedState);
}
void _EXIT ProgramExit(void)
{
}
```

Program/Variables.var

```
VAR
    DoorState : DoorStateMachine;
    EVIRD : DriveStateMachine;
    LedState : LedStateMachine;
END_VAR
```

ПРИЛОЖЕНИЕ Б

Автомат двери DoorStateMachine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
#include "Library.h"
#ifdef __cplusplus
    };
#endif
/* State mashine for door control */
void DoorStateMachine(struct DoorStateMachine* inst)
{
    int high = 250;
    int mid = 750;
    int low = 250;
    int false = 0;
    int true = 1;
    switch(inst->state)
    {
        case 0:
            {
                inst->state = 1;
                break;
            }
        case 1:
            if (inst->direction==false && inst->sw1!=inst->swp1){
                inst->state = 2;
                inst->speed = 0;
                break;
            }
            if (inst->direction==true && inst->sw4!=inst->swp4){
                inst->state = 3;
                inst->speed = 0;
                break;
            }
            break;
        case 2:
            if (inst->direction==false){
                inst->state = 4;
                inst->speed = low;
                break;
            }
            break;
        case 3:
            if (inst->direction==true){
                inst->state = 5;
                inst->speed = -low;
                break;
            }
            break;
        case 4:
            if (inst->sw2!=inst->swp2){
                inst->state = 6;
                inst->speed = mid;
                break;
            }
            if (inst->direction==true){
                inst->state = 9;
                inst->speed = -high;
                break;
            }
            break;
```

Продолжение приложения Б

```
        case 5:
            if (inst->sw3!=inst->swp3){
                inst->state = 7;
                inst->speed = -mid;
                break;
            }
            if (inst->direction==false){
                inst->state = 8;
                inst->speed = high;
                break;
            }
            break;
        case 6:
            if (inst->sw3!=inst->swp3){
                inst->state = 8;
                inst->speed = high;
                break;
            }
            if (inst->direction==true){
                inst->state = 7;
                inst->speed = -mid;
                break;
            }
            break;
        case 7:
            if (inst->sw2!=inst->swp2){
                inst->state = 9;
                inst->speed = -high;
                break;
            }
            if (inst->direction==false){
                inst->state = 6;
                inst->speed = mid;
                break;
            }
            break;
        case 8:
            if (inst->sw4!=inst->swp4){
                inst->state = 3;
                inst->speed = 0;
                break;
            }
                break;
        case 9:
            if (inst->sw1!=inst->swp1){
                inst->state = 2;
                inst->speed = 0;
                break;
            }
    }
    inst->state_out = inst->state;
    inst->swp1 = inst->sw1;
    inst->swp2 = inst->sw2;
    inst->swp3 = inst->sw3;
    inst->swp4 = inst->sw4;
    /*TODO: Add your code here*/
}
```

ПРИЛОЖЕНИЕ В

Управление преобразователем DriveStateMachine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "Library.h"
#ifdef __cplusplus
    };
#endif
/* TODO: Add your comment here */
void DriveStateMachine(struct DriveStateMachine* inst)
{
    UINT mask=inst->STATE & 0x6f;
    if (!inst->ENABLE)
    {
        inst->COM = CMD_SHUTDOWN;
    }
    else
    {
        switch (mask)
        {
            case STATE_DISABLED:
                inst->COM=CMD_SHUTDOWN;
                break;
            case STATE_READY:
                inst->COM=CMD_ENABLED;
                break;
            case STATE_SWITCHED_ON:
                inst->COM=CMD_SWITCHED_ON;
                break;
        }
    }
    /*TODO: Add your code here*/
}
```

Константы Constant.var

```
VAR CONSTANT
    CMD_SWITCHED_ON : UINT := 7;
    CMD_DISABLE_VOLTAGE : UINT := 0;
    CMD_ENABLED : UINT := 15;
    CMD_SHUTDOWN : UINT := 6;
    STATE_READY : UINT := 33;
    STATE_DISABLED : UINT := 64;
    STATE_SWITCHED_ON : UINT := 35;
END_VAR
```

ПРИЛОЖЕНИЕ Г

Автомат индикации LedStateMachine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
#include "Library.h"
#ifdef __cplusplus
    };
#endif
/* Tut mog bit vash commentariy */
void LedStateMachine(struct LedStateMachine* inst)
{
    switch(inst->state){
        case 0:
            inst->led1 = 0;
            inst->led1 = 0;
            inst->led1 = 0;
            inst->led1 = 0;
            break;
        case 1:
            if (inst->timer < 5){
                inst->led1 = 0;
                inst->led2 = 0;
                inst->led3 = 0;
                inst->led4 = 0;
                inst->timer = inst->timer + 1;
                break;
            }
            if (inst->timer < 10){
                inst->led1 = 1;
                inst->led2 = 1;
                inst->led3 = 1;
                inst->led4 = 1;
                inst->timer = inst->timer + 1;
                break;
            }
            inst->timer = 0;
            break;
```

Продолжение приложения Г

```
        case 2:
            inst->led1 = 0;
            inst->led2 = 0;
            inst->led3 = 0;
            inst->led4 = 0;
            break;
        case 3:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 1;
            inst->led4 = 1;
            break;
        case 4:
            inst->led1 = 1;
            inst->led2 = 0;
            inst->led3 = 0;
            inst->led4 = 0;
            break;
        case 5:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 1;
            inst->led4 = 0;
            break;
        case 6:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 0;
            inst->led4 = 0;
            break;
        case 7:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 0;
            inst->led4 = 0;
            break;
        case 8:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 1;
            inst->led4 = 0;
            break;
        case 9:
            inst->led1 = 1;
            inst->led2 = 0;
            inst->led3 = 0;
            inst->led4 = 0;
            break;
    }
}
```

ПРИЛОЖЕНИЕ Д

Описание интерфейсов Library.fun

```
{REDUND_ERROR} FUNCTION_BLOCK DriveStateMachine (*TODO: Add your comment here*)
    (*$GROUP=User,$CAT=User,$GROUPICON=User.png,$CATICON=User.png*)
    VAR_INPUT
        ENABLE : BOOL;
        STATE : UINT;
    END_VAR
    VAR_OUTPUT
        COM : UINT;
    END_VAR
END_FUNCTION_BLOCK
{REDUND_ERROR} FUNCTION_BLOCK DoorStateMachine (*TODO: Add your comment here*)
    (*$GROUP=User,$CAT=User,$GROUPICON=User.png,$CATICON=User.png*)
    VAR
        swp1 : BOOL := 0;
        swp2 : BOOL := 0;
        swp3 : BOOL := 0;
    END_VAR
    VAR_INPUT
        swp4 : {REDUND_UNREPLICABLE} BOOL := 0;
        sw1 : {REDUND_UNREPLICABLE} BOOL;
        sw2 : {REDUND_UNREPLICABLE} BOOL;
        sw3 : {REDUND_UNREPLICABLE} BOOL;
        sw4 : {REDUND_UNREPLICABLE} BOOL;
        direction : {REDUND_UNREPLICABLE} BOOL;
        state : INT := 0;
    END_VAR
    VAR_OUTPUT
        speed : {REDUND_UNREPLICABLE} INT;
        state_out : INT;
    END_VAR
END_FUNCTION_BLOCK
{REDUND_ERROR} FUNCTION_BLOCK LedStateMachine (*TODO: Add your comment here*)
    (*$GROUP=User,$CAT=User,$GROUPICON=User.png,$CATICON=User.png*)
    VAR_INPUT
        state : {REDUND_UNREPLICABLE} INT := 0;
    END_VAR
    VAR_OUTPUT
        led1 : {REDUND_UNREPLICABLE} BOOL;
        led2 : {REDUND_UNREPLICABLE} BOOL;
        led3 : {REDUND_UNREPLICABLE} BOOL;
        led4 : {REDUND_UNREPLICABLE} BOOL;
    END_VAR
    VAR
        timer : UINT;
    END_VAR
END_FUNCTION_BLOCK
```